

DFM-VLA: Iterative Action Refinement for Robot Manipulation via Discrete Flow Matching

Jiayi Chen^{1,2*}, Wenxuan Song^{1*}, Shuai Chen^{3,4}, Jingbo Wang¹, Zhijun Li^{2†}, Haoang Li^{1†}

¹The Hong Kong University of Science and Technology (Guangzhou), ²Harbin Institute of Technology,

³ShanghaiTech University, ⁴Shanghai Institute of Technical Physics, CAS

*Equal Contribution, †Corresponding Author

Vision–Language–Action (VLA) models that encode actions using a discrete tokenization scheme are increasingly adopted for robotic manipulation, but existing decoding paradigms remain fundamentally limited. Whether actions are decoded sequentially by autoregressive VLAs or in parallel by discrete diffusion VLAs, once a token is generated, it is typically fixed and cannot be revised in subsequent iterations, so early token errors cannot be effectively corrected later. We propose DFM-VLA, a discrete flow matching VLA for iterative refinement of action tokens. DFM-VLA models a token-level probability velocity field that dynamically updates the full action sequence across refinement iterations. We investigate two ways to construct the velocity field: an auxiliary velocity-head formulation and an action-embedding-guided formulation. Our framework further adopts a two-stage decoding strategy with an iterative refinement stage followed by deterministic validation for stable convergence. Extensive experiments on CALVIN, LIBERO, and real-world manipulation tasks show that DFM-VLA consistently outperforms strong autoregressive, discrete diffusion, and continuous diffusion baselines in manipulation performance while retaining high inference efficiency. In particular, DFM-VLA achieves an average success length of 4.44 on CALVIN and an average success rate of 95.7% on LIBERO, highlighting the value of action refinement via discrete flow matching for robotic manipulation. Our project is available <https://chris1220313648.github.io/DFM-VLA/>

Correspondence: Jiayi Chen at 1952296@tongji.edu.cn



1 Introduction

Vision–Language–Action (VLA) (Wang et al., 2025d; Black et al., 2024; Bjorck et al., 2025; Cui et al., 2025; Kim et al., 2025; Song et al., 2025c; Wang et al., 2025c; Zhong et al., 2026; Yan et al., 2026) models have become a promising foundation for robotic manipulation, where policies map language instructions and visual observations to executable actions. A common and effective design is to discretize actions into tokens, which enables scalable training with Vision-Language Models (VLMs) backbones and leverages their strong capabilities in vision and language understanding.

Most existing discrete VLA systems fall into two families. Autoregressive (AR) methods (Brohan et al., 2023; Kim et al., 2025; Wang et al., 2025d) decode tokens sequentially with a next-token prediction objective. However, their inherently left-to-right decoding order makes it difficult to revise erroneous tokens once they are emitted. Discrete diffusion (DD) methods (Song et al., 2025b; Liang et al., 2025b; Wen et al., 2025b) improve parallelism and often reduce latency, but they may still produce erroneous tokens in early iterations under confidence-guided decoding (see Figure 1). As a result, AR and DD paradigms struggle with a shared issue in robot control: early decoding errors cannot be effectively corrected and therefore propagate through the action chunk, degrading downstream robotic task performance. Recent Discrete Flow Matching (DFM) (Wang et al., 2025a; Luo et al., 2025; Havasi et al., 2025; Nguyen et al., 2025; Deng et al., 2025) studies on Large-Language Models (LLMs) or VLMs have demonstrated competitive performance on text reasoning and editing, as well as image and video generation, by enabling iterative token refinement and flexible generation trajectories

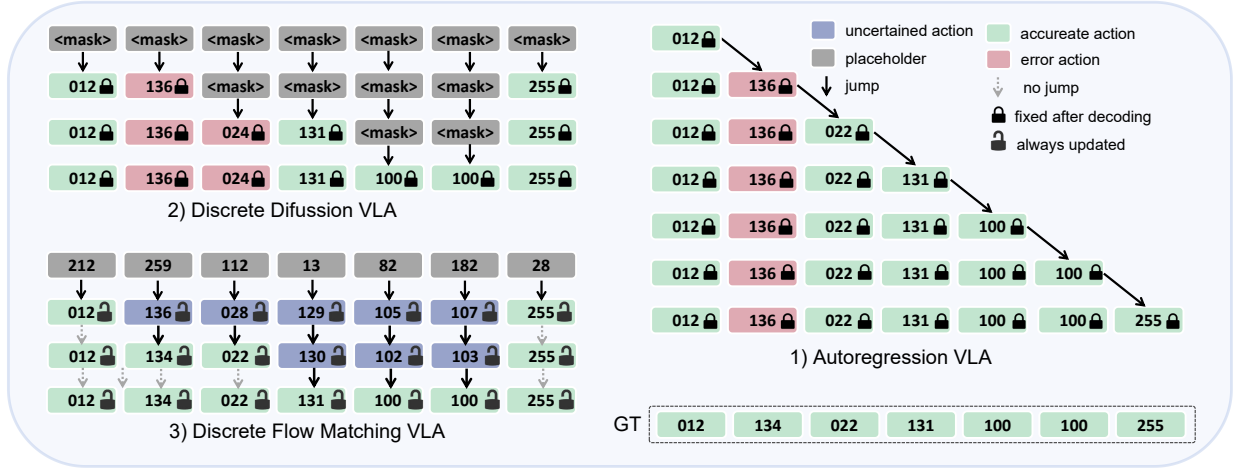


Figure 1 Comparison of decoding paradigms. 1) Autoregressive decoding requires as many steps as the action sequence length, while 2) Discrete Diffusion enables faster generation through parallel token updates. However, both have the same limitation that once an erroneous token is produced, it cannot be corrected in later iterations. We refer to this phenomenon as *irreversible commitment*. In contrast, 3) our DFM-VLA performs full-sequence action refinement at every iteration, allowing token-level correction and improving action quality for robotic manipulation.

guided by velocity field. These advances inspire us to transfer the same refinement principle to discrete action generation for robotic control.

We propose DFM-VLA, a VLA framework that unifies language, vision, and action in a discrete formulation, and employs a discrete action velocity field to enable holistic refinement of action sequences. As shown in Figure 1, instead of treating action prediction as one-shot token generation, DFM-VLA models a token-level probability velocity field and performs iterative full-sequence refinement. This formulation allows the model to repeatedly revisit previously updated positions and selectively correct uncertain tokens as context becomes more informative. For velocity-field construction, we explore two variants, namely an auxiliary velocity head formulation that predicts velocities from model hidden states, and an action-embedding-guided formulation that defines semantically structured probability paths and derives kinetic-optimal velocities. For decoding, we adopt a two-stage strategy consisting of an iterative refinement stage for exploration and correction, followed by a deterministic validation stage for stable convergence.

We evaluate DFM-VLA on CALVIN, LIBERO, and real-world manipulation tasks. Across benchmarks, DFM-VLA consistently improves robot manipulation quality while maintaining strong inference efficiency. Ablation studies further indicate that the proposed design is robust to key scheduler settings, training data scale, and two-stage decoding allocation.

Our contributions are summarized as follows:

- We identify the *irreversible commitment* problem in existing autoregressive and discrete diffusion VLA decoding, which limits token correction in robot manipulation.
- We propose DFM-VLA, a discrete flow matching VLA framework that performs iterative full-sequence action refinement via token-level velocity.
- We propose two strategies for velocity field construction, namely an auxiliary velocity head formulation and an action-embedding-guided formulation, and we systematically analyze these design variants.
- We demonstrate strong empirical performance on CALVIN, LIBERO, and real-world tasks, with consistent gains in both action quality and inference efficiency.

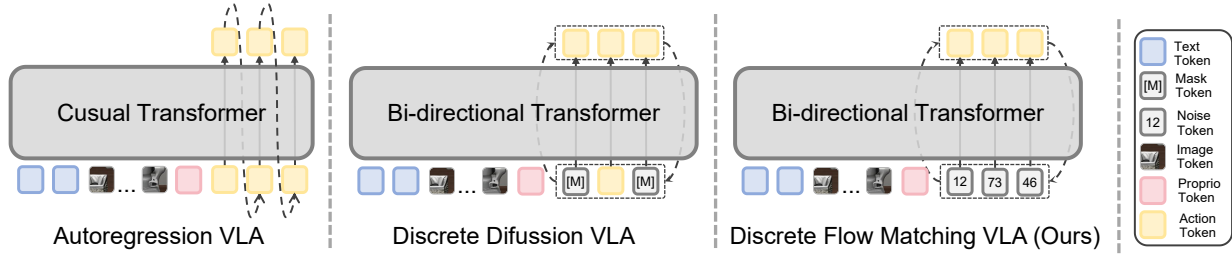


Figure 2 Comparison of three discrete VLA paradigms. AR VLA uses causal attention to generate future action tokens from previously generated ones, DD-VLA decodes only masked tokens, while DFM-VLA iteratively refines tokens over the full sequence.

2 Related Works

Autoregressive VLA. Autoregressive VLA models (see Figure 2) represent actions as discrete tokens, enabling LLM-style decoding and scalable training. Representative systems include RT-1 (Brohan et al., 2023), OpenVLA (Kim et al., 2025), and LLaVA-VLA (Song et al., 2026), which unify vision, language, and action prediction under the next-to-next objective. ReconVLA (Song et al., 2025c) improves perception by reconstructing the current frame to assist discrete action generation. Meanwhile, dynamic inference or efficiency tokenizer designs (e.g., Deer-VLA (Yue et al., 2024) and FAST (Pertsch et al., 2025)) target practical deployment constraints. Methods based on vector quantization (VQ) (Wang et al., 2025b; Liu et al., 2025; Dong et al., 2026) provide a flexible alternative by learning discrete latent representations. UniVLA (Wang et al., 2025d) and WorldVLA (Cen et al., 2025) discretize future frames to leverage dynamic visual information for action generation during both training and inference. These autoregressive models are effective but inherently sequential, which motivates our refinement alternative.

Discrete Diffusion VLA. Discrete diffusion VLA models extend dLLM-style (Yu et al., 2025) parallel denoising to action sequences, enabling full-attention, multi-token decoding with iterative refinement. Representative methods differ in their corruption processes and efficiency strategies. PD-VLA (Song et al., 2025b) follows a BART-style noising scheme by substituting tokens with vocabulary items and learning to reconstruct the sequence, while Discrete Diffusion VLA (Liang et al., 2025b) and LLADA-VLA (Wen et al., 2025b) adopt BERT-style masking with a dedicated mask token (Devlin et al., 2019). To reduce inference cost, CEED-VLA (Song et al., 2025a) applies consistency distillation to shrink the number of denoising steps with minimal performance loss. On the data and modeling side, Dream-VLA (Ye et al., 2025b) scales discrete diffusion pretraining on the OXE (O’Neill et al., 2024) datasets following OpenVLA (Kim et al., 2025). Likewise, UD-VLA (Chen et al., 2025), dVLA (Wen et al., 2025a), and MM-ACT (Liang et al., 2025a) incorporate visual or textual chain-of-thought (CoT) and jointly diffuse future frames, reasoning traces, and action tokens for unified perception–reasoning–action generation. However, these dVLA methods cannot perform token-level refinement, as their updates are restricted to masked positions rather than allowing full-sequence refinement.

Discrete Flow Matching LLM and VLM. Discrete flow matching (Gat et al., 2024) provides a principled framework for modeling probability paths over discrete tokens, with theoretical work (Shaul et al., 2024) establishing kinetic-optimal discrete paths and velocity formulations that generalize mask-based mixtures. Building on these foundations, several large-scale multimodal models adopt discrete flow matching for unified understanding and generation, including Fudoki (Wang et al., 2025a) and Next-Omni (Luo et al., 2025), demonstrating strong any-to-any or omnimodal capabilities. EditFlow (Havasi et al., 2025) introduces explicit edit operations (insertion, replacement, deletion) with learned rates to enable flexible sequence refinement, while OneFlow (Nguyen et al., 2025) extends this idea to concurrent mixed-modal and interleaved generation. URSA (Deng et al., 2025) further illustrates how discrete metric structures can guide token transitions in generative modeling. We introduce discrete flow matching to the action modality, leveraging iterative refinement via a velocity field to progressively enhance action generation.

3 Preliminary: Discrete Flow Matching

We briefly review the key concepts and notation of discrete flow matching (Gat et al., 2024; Wang et al., 2025a) that are used throughout the paper. DFM aims to transform a known source distribution $p(x)$ into a target data distribution $q(x)$ over a discrete space. We consider $x = (x^1, x^2, \dots, x^D) \in \mathcal{S} = \mathcal{T}^D$, where D is the number of discrete variables and $\mathcal{T} = [K] = \{1, 2, \dots, K\}$ is the finite alphabet of possible values.

Probability Paths. Given a *source distribution* $p(x)$ and a *target distribution* $q(x)$ on a finite state space $\mathcal{S}$, DFM defines a family of time-indexed distributions $\{p_t(x)\}_{t \in [0,1]}$ that smoothly interpolates between p and q , referred to as *probability paths*. Each $p_t(x)$ is constructed as $p_t(x) := \sum_{x_1 \in \mathcal{S}} p_t(x | x_1) q(x_1)$, where the conditional distribution factorizes across dimensions, i.e., $p_t(x | x_1) := \prod_{i=1}^D p_t(x^i | x_1^i)$. Each term $p_t(x^i | x_1^i)$ interpolates between the base distribution $p(x^i)$ and a point mass $\delta_{x_1^i}(x^i)$, i.e., $\delta_{x_1^i}(x^i) = 1$ if $x^i = x_1^i$ and 0 otherwise. A common choice is the *mixture path* used in (Gat et al., 2024; Havasi et al., 2025), defined by a time-dependent scheduler $\kappa_t(x_1^i) \in [0, 1]$:

$$p_t(x^i | x_1^i) = (1 - \kappa_t(x_1^i))p(x^i) + \kappa_t(x_1^i)\delta_{x_1^i}(x^i), \quad (1)$$

where $\kappa_0(\cdot) = 0$ and $\kappa_1(\cdot) = 1$. The term $p_t(x^i | x_1^i)$ is a *conditional forward probability path* that describes how the state x^i evolves given x_1^i .

Probability velocities. To realize the prescribed probability path $p_t(x)$, we use a Continuous-Time Markov Chain (CTMC). Its dynamics are governed by a probability velocity u_t , also called the *transition rate*, which describes how the current state x_t moves toward the target state x_1 over time. Each token i is updated independently according to

$$x_{t+h}^i \sim \delta_{x_t^i}(\cdot) + h u_t^i(\cdot | x_t^i, x_1^i), \quad (2)$$

where $u_t^i(\cdot | x_t^i, x_1^i)$ is the *velocity field*, a conditional rate function that governs the flow of probability from x_t^i to x_1^i . Equation (2) can be viewed as a small perturbation of the point mass $\delta_{x_t^i}$ scaled by the step size h , which models discrete state transitions in continuous time. The velocity field is central to DFM, which defines the probability-path dynamics and is the main quantity learned during training.

4 Method

This section is organized into four parts. We first introduce the model architecture in Sec. 4.1. We then present two velocity-field constructions: 1) an auxiliary velocity head formulation in Sec. 4.2 and 2) an embedding-guided formulation in Sec. 4.3. Finally, we describe the two-stage inference procedure in Sec. 4.4.

4.1 Architecture

As shown in Figure 3, DFM-VLA follows an architecture similar to UniVLA and adopts a unified discrete token formulation for language, vision, and action modalities. Specifically, we tokenize text using the Emu3 (Wang et al., 2024) tokenizer, and discretize visual observations with a VQ tokenizer (Zheng et al., 2022) using a compression ratio of 4. Both third-person and wrist-view images are represented as $25 \times 25 = 625$ tokens. For action encoding, we discretize continuous actions with FAST (Pertsch et al., 2025) and further compress action tokens using Byte Pair Encoding (BPE), with an action vocabulary size of 1024. To explicitly distinguish modalities, we enclose image and action token spans with boundary markers, i.e.,

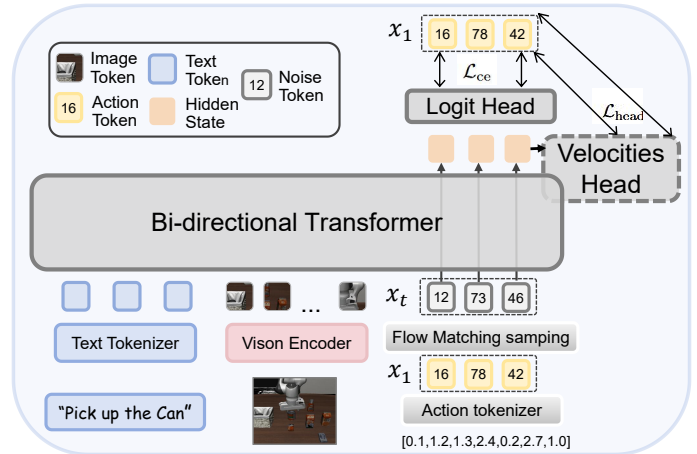


Figure 3 Overall architecture of DFM-VLA. Given language-vision context and noised action tokens x_t , the model predicts clean actions x_1 and learns the velocity field via $\mathcal{L}_{ce}$ or $\mathcal{L}_{head}$.

boi/eoi for begin/end of image and boa/ea for begin/end of action. In this work, we denote the discretized instruction and observations as l , and apply noising and prediction only to the action modality.

4.2 Modeling Velocities by Additional Head

Drawing inspiration from EditFlow (Havasi et al., 2025), which employs additional velocity heads to model three types of edit operations, namely *insertion*, *replacement*, and *deletion*, we explore an alternative formulation for velocity-field construction. We retain only the *replacement* operation, because the action token sequence length is predefined by our action chunk design and the three operations are fundamentally equivalent under appropriate reformulation (Nguyen et al., 2025; Havasi et al., 2025). Concretely, given noisy action tokens x_t and context l , the backbone first produces hidden states, and the additional velocity head then maps these hidden states to replacement velocities:

$$h_t = f_\theta(x_t, l), \quad u_t^\theta(\cdot | x_t) = u_t^{\text{head}}(h_t), \quad (3)$$

where f_θ denotes the backbone network and u_t^{head} denotes the velocity prediction head.

$$\mathcal{L}_{\text{head}} = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1, x_t} \left[\sum_{x \neq x_t} u_t^\theta(x | x_t) - \sum_{i=1}^N \mathbf{1}_{[x_t^i \neq x_1^i]} \log u_t^\theta(x^i | x_t^i) p_{1|t}(x_1^i | x_t^i, l) \right]. \quad (4)$$

Here, $\mathbf{1}_{[x_t^i \neq x_1^i]}$ is an indicator that equals 1 only when the current token differs from the target token, so the update loss is applied only to positions that still need refinement.

4.3 Action-Embedding-Guided Velocity Modeling

Building on recent advances in discrete flow matching (Gat et al., 2024; Wang et al., 2025a; Deng et al., 2025), we parameterize the probability path in a metric-induced form. Specifically, let $d : \mathcal{T} \times \mathcal{T} \rightarrow \mathbb{R}_{\geq 0}$ be a distance such that $d(x^i, x_1^i) = 0$ if and only if $x^i = x_1^i$, where $d(\cdot, \cdot)$ is measured in the action token embedding space. We define the conditional path as

$$p_t(x^i | x_1^i) = \text{softmax}(-\beta_t \cdot d(x^i, x_1^i)), \quad (5)$$

where $\beta_t : [0, 1] \rightarrow \mathbb{R}_{\geq 0}$ is a monotonic schedule with boundary conditions $\beta_0 = 0$ and $\beta_1 = \infty$. We instantiate it as

$$\beta_t = c \left(\frac{t}{1-t} \right)^\alpha, \quad t \in [0, 1), \quad (6)$$

where $c > 0$ and $\alpha > 0$ control how fast probability mass concentrates toward the target token over time. This formulation preserves semantic neighborhood structure such that tokens closer to x_1^i receive larger probability as $t \rightarrow 1$.

Given this prescribed path, we adopt the kinetic-optimal velocity obtained by minimizing transport energy under the flow constraints (Wang et al., 2025a; Deng et al., 2025; Luo et al., 2025):

$$u_t^i(x^i, z | x_1) = p_t(x^i | x_1^i) \dot{\beta}_t [d(z^i, x_1^i) - d(x^i, x_1^i)]_+ \quad (7)$$

where $[\cdot]_+ = \max\{\cdot, 0\}$, z^i is token in the vocabulary $\mathcal{T}$, and $\dot{\beta}_t$ is the derivative of β_t w.r.t. t . This velocity moves probability mass from z^i to x^i only when x^i is closer to x_1^i than z^i , yielding a monotonic refinement process toward the clean target token.

Training. Given language-vision context l and corrupted action tokens x_t , the model predicts the target action sequence x_1 by outputting per-position categorical logits. We optimize the expected cross-entropy:

$$\mathcal{L}_{\text{ce}} = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1, x_t} [-\log p_{1|t}(x_1 | x_t, l)]. \quad (8)$$

where $p_{1|t}^\theta(\cdot | x_t, l)$ denotes the predicted categorical distribution of the model at each action token position.

As shown in Figure 5, the embedding-guided objective ($\mathcal{L}_{\text{ce}}$) converges faster and achieves better final performance than the head-based objective ($\mathcal{L}_{\text{head}}$). Unless otherwise specified, all experiments use the embedding-guided setting.

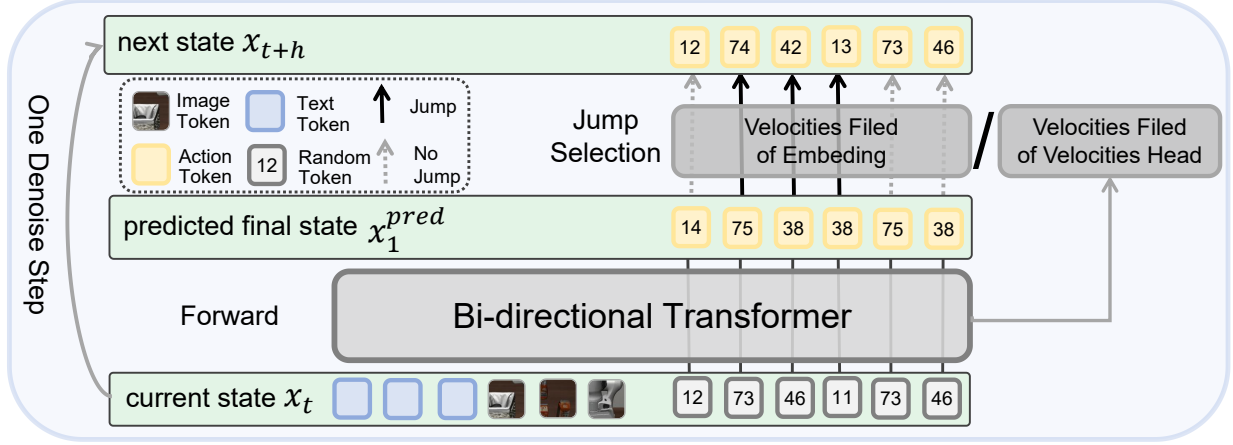


Figure 4 Visualization of a single decoding step in the iterative refinement stage. After predicting final state x_1^{pred} , the model does not directly output final action tokens. Instead, it constructs a velocity field to compute transition rates and selectively updates tokens to next state x_{t+h} at each step.

4.4 Inference

We perform inference in two stages: an iterative refinement stage followed by a deterministic validation stage, with T_{fine} and T_{val} decoding steps, respectively.

In the iterative refinement stage, we employ an Euler discretization of the continuous-time Markov chain (CTMC) process $(x_t)_{0 \leq t \leq 1}$, following the approach in (Deng et al., 2025). As illustrated in Figure 4, for each coordinate i and each step from t to $t+h$, we perform:

- Sample $x_1^i \sim p_{1|t}^i(\cdot | x_t, l)$ from the model;
- Compute the total outgoing rate $\lambda^i = \sum_{x^i \neq x_t^i} u_t^i(x^i, x_t^i | x_1^i)$ using Eq. 7 or Eq. 3;
- Draw $Z_{\text{change}}^i \sim U[0, 1]$;
- Update x_{t+h}^i : if $Z_{\text{change}}^i \leq 1 - e^{-h\lambda^i}$, sample from $\frac{u_t^i(\cdot, x_t^i | x_1^i)}{\lambda^i} (1 - \delta_{x_t^i}(\cdot))$; otherwise keep $x_{t+h}^i = x_t^i$.

Here, λ^i is the total transition intensity out of the current token x_t^i , so the jump probability $1 - e^{-h\lambda^i}$ increases with λ^i . When a jump occurs, normalized rates favor states with larger velocity flow, which typically move the token closer to the predicted clean state x_1^i . Repeating this process over time enables iterative correction across the full sequence. Compared with mask-based discrete diffusion methods (Ye et al., 2025a), this decoding scheme allows previously updated tokens to be revised again in later iterations, rather than being permanently fixed after being predicted.

During the validation stage, we adopt a greedy decoding strategy to improve stability in the final refinement steps. In the final T_{val} decoding steps, we disable stochastic jumps and switch to greedy decoding,

$$x_{t+h} = \arg \max \text{softmax}(p_{1|t}(\cdot | x_t)), \quad (9)$$

This hybrid design preserves exploratory refinement in earlier iterations while enforcing deterministic convergence near the end, leading to more stable validation-time trajectories and improved reproducibility. The concise two-stage decoding procedure is summarized in Algorithm 1.

Adaptive KV Caching. Following the dynamic caching strategy of FAST-dLLM (Wu et al., 2025) on discrete diffusion decoding, we exploit the fact that many tokens exhibit only minor KV-state changes across iterative denoising steps of DFM. We keep the instruction and observation KV caches largely fixed throughout inference, while adaptively updating the action-side cache based on the cosine similarity between current and cached value features. Combined with the parallel refinement of DFM-VLA, this dynamic KV reuse yields a $2.4 \times$ latency speedup over autoregressive decoding while preserving task performance (see Table 5).

Algorithm 1 Two-Stage Decoding of DFM-VLA

```

1: Input: predictor  $p_\theta$ , context  $l$ , steps  $T_{\text{fine}}, T_{\text{val}}$ , action vocabulary  $\mathcal{V}$ 
2: Sample  $x_0 \sim \text{Uniform}(\mathcal{V})$ ; set  $T \leftarrow T_{\text{fine}} + T_{\text{val}}$ 
3: for  $k = 1$  to  $T$  do
4:    $t \leftarrow (k - 1)/T$ ,  $h \leftarrow 1/T$ 
5:    $\hat{x}_1 \sim p_\theta(\cdot \mid x_t, l)$ 
6:   if  $k \leq T_{\text{fine}}$  then
7:     Compute velocity  $u_t$  from  $\hat{x}_1$  (Eq. 7 or Eq. 3)
8:     Update  $x_{t+h}$  by a CTMC Euler step
9:   else
10:    Update  $x_{t+h} \leftarrow \arg \max p_\theta(\cdot \mid x_t, l)$ 
11:   end if
12: end for
13: Output: action sequence  $x_1$ 

```

Table 1 Comprehensive Evaluation of Long-Horizon Robotic Manipulation on the CALVIN Benchmark. UniVLA* denotes the variant without historical frames for fair comparison.

Method	Task	Tasks Completed in a Row					Avg. Len $\uparrow$
		1	2	3	4	5	
MCIL (Lynch and Sermanet, 2020)	ABCD $\rightarrow$ D	0.373	0.027	0.002	0.000	0.000	0.40
RT-1 (Brohan et al., 2023)	ABCD $\rightarrow$ D	0.844	0.617	0.438	0.323	0.227	2.45
Robo-Flamingo (Li et al., 2024)	ABCD $\rightarrow$ D	0.964	0.896	0.824	0.740	0.660	4.09
Deer (Yue et al., 2024)	ABCD $\rightarrow$ D	0.982	0.902	0.821	0.759	0.670	4.13
GR-1 (Wu et al., 2024)	ABCD $\rightarrow$ D	0.949	0.896	0.844	0.789	0.731	4.21
ReconVLA (Song et al., 2025c)	ABCD $\rightarrow$ D	0.980	0.900	0.845	0.785	0.705	4.23
UniVLA* (Wang et al., 2025d)	ABCD $\rightarrow$ D	0.948	0.906	0.862	0.834	0.690	4.26
MODE (Reuss et al., 2025)	ABCD $\rightarrow$ D	0.971	0.925	0.879	0.835	0.779	4.39
UP-VLA (Zhang et al., 2025a)	ABCD $\rightarrow$ D	0.962	0.921	0.879	0.842	0.812	4.42
DFM-VLA+Head	ABCD $\rightarrow$ D	0.968	0.928	0.880	0.864	0.776	4.42
DFM-VLA+Embed	ABCD $\rightarrow$ D	0.976	0.944	0.892	0.844	0.780	4.44

5 Experiments

We conduct comprehensive experiments to evaluate the effectiveness of DFM-VLA on both simulation benchmarks and real-world robotic manipulation tasks. Our experiments are designed to answer the following research questions:

(RQ1) How does DFM-VLA compare with recent state-of-the-art VLA methods on CALVIN and LIBERO benchmarks (see Sections 5.2 and 5.3)?

(RQ2) What empirical insights can guide key design choices for DFM-VLA, including schedule hyperparameters c and α , two-stage decoding allocation, and velocity-field construction (see Section 5.4)?

(RQ3) What additional insights can we gain from in-depth analysis of decoding behavior, execution quality, and efficiency trade-offs (see Section 5.5)?

(RQ4) Can DFM-VLA generalize effectively to real-world manipulation tasks (see Section 5.6)?

5.1 Setup

We initialize our model from checkpoints pretrained on robotic video data (Wang et al., 2025d) and keep the remaining settings consistent with UniVLA. Unless otherwise specified, we use a learning rate of 1×10^{-4} and a batch size of 8. All training and inference are conducted on 8 NVIDIA H100 GPUs. For simulation benchmarks (CALVIN and LIBERO), we train for 20k–32k steps depending on the setting, while each real-world task is

Table 2 Evaluation and comparison on the LIBERO benchmark.

Method	Spatial	Object	Goal	Long	Average
Octo (Octo Team et al., 2024)	78.9%	85.7%	84.6%	51.1%	75.1%
SpatialVLA (Qu et al., 2025)	88.2%	89.9%	78.6%	55.5%	78.1%
CoT-VLA (Zhao et al., 2025)	87.5%	91.6%	87.6%	69.0%	81.1%
WorldVLA (Cen et al., 2025)	87.6%	96.2%	83.4%	60.0%	81.8%
ThinkAct (Huang et al., 2025)	88.3%	91.4%	87.1%	70.9%	84.4%
π_0 -FAST (Pertsch et al., 2025)	96.4%	96.8%	88.6%	60.2%	85.5%
MolmoAct (Lee et al., 2025)	87.0%	95.4%	87.6%	77.2%	86.6%
FlowVLA (Zhong et al., 2025)	93.2%	95.0%	91.6%	72.6%	88.1%
DreamVLA (Zhang et al., 2025b)	97.5%	94.0%	89.5%	89.5%	92.6%
DFM-VLA+Head	94.2%	96.4%	92.8%	90.4%	93.5%
DFM-VLA+Embed	96.8%	98.8%	94.4%	92.6%	95.7%

trained for 5k steps. Unless otherwise noted, we follow (Luo et al., 2025) and use $c = 3$ and $\alpha = 1$ as the default setting.

5.2 Benchmarks

CALVIN. CALVIN (Mees et al., 2022) is a simulation benchmark for long-horizon, language-conditioned robotic manipulation, covering four environments (A, B, C, and D), 34 manipulation skills, and 1,000 language annotations. In this work, we follow the standard **ABCD**→**D** setup. Concretely, each evaluation rollout contains a chain of five consecutive language-conditioned sub-tasks, and success on later sub-tasks depends on the correctness of earlier actions. This makes CALVIN particularly suitable for measuring error accumulation and temporal consistency in multi-step control. We evaluate 1,000 rollouts per model and report the average number of consecutively completed sub-tasks (Avg. Len., maximum 5), together with per-step completion rates.

LIBERO. LIBERO (Liu et al., 2023) is a simulation benchmark designed to stress-test cross-task generalization along multiple dimensions. It is organized into four suites: Spatial, Object, Goal, and Long. Spatial focuses on spatial-relation understanding under varied layouts, Object emphasizes object-level generalization, Goal evaluates goal-conditioned manipulation under different target configurations, and Long tests long-horizon compositional reasoning over multi-stage behaviors. Compared with CALVIN, LIBERO places stronger emphasis on broad generalization across task families. We report per-suite success rates and the overall average. Each suite contains 10 tasks, with 50 evaluation rollouts per task.

5.3 Comparison with SOTA

We report the main results on CALVIN and LIBERO in Tables 1 and 2. On CALVIN, both variants of DFM-VLA outperform most strong VLA baselines, and DFM-VLA+Embed achieves the best overall average length of 4.44. Compared with unified AR VLAs (e.g., UniVLA*), DFM-VLA+Embed improves 3-step and 5-step completion (0.892 vs. 0.862 and 0.780 vs. 0.690), indicating stronger long horizon consistency. It also outperforms continuous diffusion VLA models across all reported metrics, showing that discrete action refinement is competitive with optimization in continuous action spaces. Moreover, compared with AR baselines that enhance perception, such as ReconVLA, our method delivers gains of 0.19, showing that focused action iterative refinement is more effective. Although DFM-VLA+Head is slightly weaker than DFM-VLA+Embed, it still surpasses other baselines, further supporting the robustness of velocity guided action regeneration in long-horizon tasks.

On LIBERO, our method shows clear gains across all suites. DFM-VLA+Embed reaches the best overall average success rate of 95.7%, surpassing reasoning-guided discrete VLAs that rely on text or spatial chain of thought cues, for example, ThinkAct, MolmoAct, and CoT-VLA. It also improves over DreamVLA, which follows continuous diffusion with future visual prediction, by +3.1 points, and over FlowVLA, which uses autoregressive future optical flow cues, by +7.6 points. In particular, it achieves the highest scores on Object

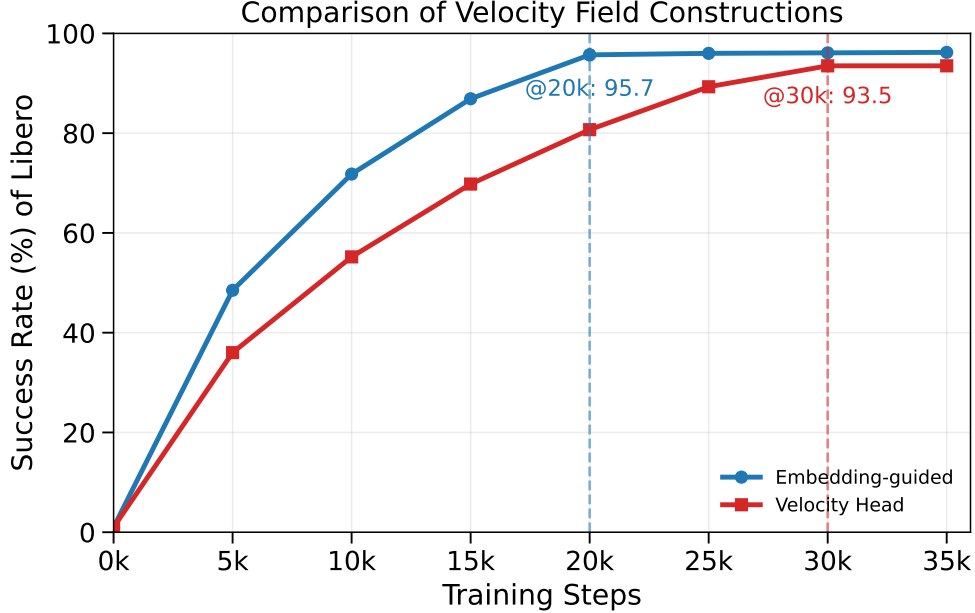


Figure 5 Comparison of velocity field constructions across training steps. The Embedding-guided variant consistently outperforms the Velocity Head variant in both convergence speed and final performance, reaching a state-of-the-art success rate of 95.7% on LIBERO with only 20k training steps.

at 98.8% and Long at 92.6%, while maintaining strong performance on Spatial and Goal tasks. These results suggest that directly refining actions with previously generated coarse action signals is more effective than relying on auxiliary modality chain of thought guidance, leading to better long-horizon execution and cross-task generalization.

5.4 Ablation Studies

We conduct ablation studies to isolate the effects of key design and training choices, including schedule hyperparameters c and α , two-stage decoding step allocation, and velocity-field construction. Detailed quantitative results are summarized in [Tables 3](#) and [4](#) and [figure 5](#).

Effect of c and α . In Eq. 6, c controls the overall scale of β_t , while α controls the temporal curvature of the schedule, determining how quickly refinement concentrates as $t \rightarrow 1$. [Table 3](#) shows that DFM-VLA is robust to moderate changes of both parameters, with $c = 3, \alpha = 1$ giving the best trade-off. Under this setting, DFM-VLA achieves the Avg. Len. of 4.44 on CALVIN and average success rate of 95.7% on Libero. Increasing c to 5 or decreasing α to 0.5 causes only minor fluctuations, indicating stable refinement dynamics across a reasonably wide hyperparameter region.

Table 3 Ablation study on c and α .

c	α	CALVIN $\uparrow$	LIBERO (%) $\uparrow$
1	1	4.38	95.0
3	1	4.44	95.7
5	1	4.40	95.2
3	0.5	4.40	95.6

Effect of Two-Stage Decoding. We further ablate how decoding steps are allocated between the iterative refinement stage (T_{fine}) and the deterministic validation stage (T_{val}). We keep the total number of steps fixed at 16 and report Avg. Len. of CALVIN ABCD $\rightarrow$ D and average success rate of LIBERO in [Table 4](#). As shown in the table, using only iterative refinement ($T_{\text{val}} = 0$) yields weaker

Table 4 Ablation on two-stage decoding step allocation with fixed total steps ($T_{\text{fine}} + T_{\text{val}} = 16$).

T_{fine}	T_{val}	CALVIN $\uparrow$	LIBERO (%) $\uparrow$
16	0	4.37	94.8
15	1	4.39	95.2
14	2	4.44	95.7
12	4	4.41	95.4
8	8	4.35	95.1

Table 5 Inference efficiency comparison across AR, DD, and DFM on CALVIN ABCD→D. DD and DFM use the same decoding steps (16).

Method	Avg. Len. ↑	Speed ↑
AR	4.18	50.2
DD	4.32	62.1
DD + Adap. Cache	4.27	118.3
DFM	4.42	60.2
DFM + Adap. Cache	4.40	121.0

Table 6 Ablation on training data scale on CALVIN ABCD→D. We compare three discrete VLA paradigms across data scales, with DD and DFM evaluated using the same number of decoding steps.

Data Fraction	AR	DD	DFM (Ours)
10%	1.71	2.84	3.21
50%	3.01	3.88	4.03
100%	4.18	4.32	4.44

performance. Introducing a short validation stage improves both benchmarks, and $T_{\text{fine}} = 14, T_{\text{val}} = 2$ achieves the best overall trade-off. In contrast, allocating too many steps to the validation stage slightly hurts action quality, suggesting that excessive early greedy release reduces refinement flexibility. Therefore, we use $T_{\text{fine}} = 14$ and $T_{\text{val}} = 2$ in all experiments.

Comparison of Velocity Field Constructions. To further analyze the effect of velocity-field design, we compare the embedding-guided formulation with the head-based formulation across training steps. As shown in Figure 5, the embedding-guided variant converges faster in the early stages of training and consistently reaches better final task performance. This trend indicates that embedding guidance provides more informative and smooth optimization signals, leading to both improved data efficiency and a stronger final policy.

5.5 In-Depth Analysis

Effectiveness of Decoding Method. We keep the architecture unchanged and compare AR, DD, and our method, while also validating the effectiveness of Adaptive Cache in inference. Results in Table 5 show that DFM-VLA achieves the best action quality among the compared decoding strategies. With Adaptive Cache enabled, both DD and DFM obtain clear speedups, and *DFM+Adaptive Cache* delivers the fastest decoding while maintaining strong performance. These results indicate that DFM provides a favorable quality–efficiency trade-off under matched decoding steps.

Effect of training data size. Table 6 shows that DFM-VLA consistently outperforms both autoregressive and discrete diffusion baselines across all data scales on CALVIN ABCD→D. At 10% data, DFM-VLA achieves an Avg. Len. of 3.21, outperforming AR at 1.71 and discrete diffusion at 2.84 by +1.50 and +0.37, respectively. It also remains the best at 50% data with 4.03 and at 100% data with 4.44, showing consistent gains as data scale increases. These results indicate that DFM-VLA is particularly beneficial in low-data regimes while maintaining consistent advantages as training data scales up.

5.6 Real World Experiment

Setup. As shown in Figure 6, we conduct real-world experiments on a bimanual AgileX platform equipped with two robotic arms, each with six degrees of freedom and a parallel gripper. The system is instrumented with three RGB cameras: one fixed camera mounted at an elevated central viewpoint and two wrist-mounted cameras, one on each arm.

Task Setting. We design three representative manipulation tasks: (1) bimanual collaborative lifting of a pot (*Pot Lift*); (2) grasping elongated vegetables and placing them into the pot, where the object position and pose are varied (*Place Veg. to Pot*); and (3) grasping a block and placing it onto a plate with varying height (*Place Block to Plate*). For each task, we collect 100 trajectories for training. During evaluation, we run 40 trials per task and report both task success rate and completion speed.

Results. We compare DFM-VLA against representative methods from three action-generation paradigms for real-world manipulation: the autoregressive baseline π_0 -FAST, the discrete diffusion baseline Dream-VLA, and the continuous diffusion baseline RDT (Liu et al., 2024). Table 7 shows that DFM-VLA achieves the highest success rate on each task. Specifically, our DFM-VLA reaches a success rate of 77.5% on *Pot Lift*, which requires precise bimanual coordination; 70.0% on *Place Veg. to Pot*, which tests generalization to object

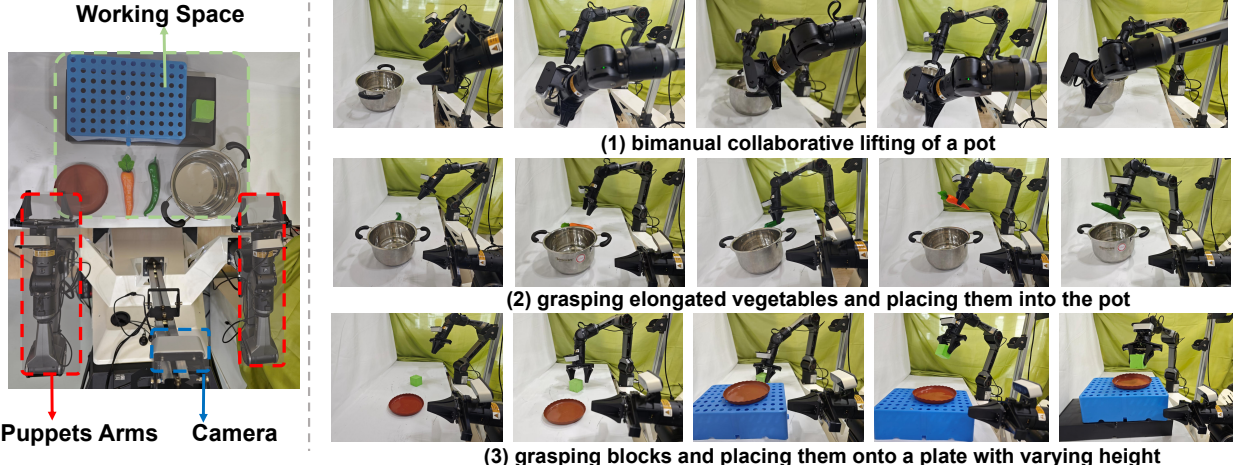


Figure 6 Left: The real-world robotic system for experiments. Right: The representative task demonstrations.

Table 7 Real-world success rate comparison (%) on three tasks.

Method	Pot Lift	Place Veg. to Pot	Place Block to Plate	Average
π_0 -FAST (Pertsch et al., 2025)	50.0	42.5	35.0	42.5
Dream-VLA (Ye et al., 2025a)	57.5	62.5	42.5	54.2
RDT (Liu et al., 2024)	65.0	62.5	52.5	60.0
DFM-VLA+Head	70.0	67.5	60.0	65.8
DFM-VLA+Embed	77.5	70.0	65.0	70.8

positions and poses; and 65.0% on *Place Block to Plate*, which emphasizes spatial precision under varying plate heights. Overall, the average success rate of DFM-VLA is 70.8%, which surpasses RDT at 60.0% by 10.8 percentage points and Dream-VLA at 54.2% by 16.6 percentage points.

This result demonstrates that traditional discrete baselines (i.e., π_0 -FAST and Dream-VLA) are weaker than the continuous diffusion baseline RDT because wrongly decoded action tokens cannot be refined, leading to error accumulation. In contrast, our method uses refined action guided by velocities over discrete action tokens to correct errors in time. Meanwhile, its unified discrete input-output space better preserves semantic understanding from autoregressive VLM backbones (Liang et al., 2025b; Wen et al., 2025a), yielding the best overall performance. These results further show that action refinement with discrete flow matching remains highly competitive and robust in real-world settings.

6 Conclusion

In this paper, we present DFM-VLA, a discrete flow-matching VLA framework that enables iterative full-sequence action refinement through token-level velocity modeling. Our DFM-VLA can revisit and correct previously generated action tokens, improving action consistency for robotic manipulation. We further study two complementary velocity-field constructions and integrate a practical two-stage decoder. Extensive experiments demonstrate that DFM-VLA consistently delivers strong action quality while maintaining high inference efficiency.

References

- Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom,

- Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. Robotics: Science and Systems Foundation, 2023.
- Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- Jiayi Chen, Wenxuan Song, Pengxiang Ding, Ziyang Zhou, Han Zhao, Feilong Tang, Donglin Wang, and Haoang Li. Unified diffusion vla: Vision-language-action model via joint discrete denoising diffusion process. *arXiv preprint arXiv:2511.01718*, 2025.
- Can Cui, Pengxiang Ding, Wenxuan Song, Shuanghao Bai, Xinyang Tong, Zirui Ge, Runze Suo, Wanqi Zhou, Yang Liu, Bofang Jia, et al. Openhelix: A short survey, empirical analysis, and open-source dual-system vla model for robotic manipulation. *arXiv preprint arXiv:2505.03912*, 2025.
- Haoqiang Deng, Ting Pan, Fan Zhang, Yang Liu, Zhuoyan Luo, Yufeng Cui, Wenxuan Wang, Chunhua Shen, Shiguang Shan, Zhaoxiang Zhang, et al. Uniform discrete diffusion with metric path for video generation. *arXiv preprint arXiv:2510.24717*, 2025.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- Zibin Dong, Yicheng Liu, Shiduo Zhang, Baijun Ye, Yifu Yuan, Fei Ni, Jingjing Gong, Xipeng Qiu, Hang Zhao, Yinchuan Li, et al. Actioncodec: What makes for good action tokenizers. *arXiv preprint arXiv:2602.15397*, 2026.
- Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky TQ Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, pages 133345–133385, 2024.
- Marton Havasi, Brian Karrer, Itai Gat, and Ricky TQ Chen. Edit flows: Flow matching with edit operations. *arXiv preprint arXiv:2506.09018*, 2025.
- Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.
- Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.
- Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. 2024.
- Haotian Liang, Xinyi Chen, Bin Wang, Mingkan Chen, Yitian Liu, Yuhao Zhang, Zhanxin Chen, Tianshuo Yang, Yilun Chen, Jiangmiao Pang, et al. Mm-act: Learn from multimodal parallel generation to act. *arXiv preprint arXiv:2512.00975*, 2025a.
- Zhixuan Liang, Yizhuo Li, Tianshuo Yang, Chengyue Wu, Sitong Mao, Tian Nian, Liyao Pei, Shunbo Zhou, Xiaokang Yang, Jiangmiao Pang, et al. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. *arXiv preprint arXiv:2508.20072*, 2025b.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- Yicheng Liu, Shiduo Zhang, Zibin Dong, Baijun Ye, Tianyuan Yuan, Xiaopeng Yu, Linqi Yin, Chenhao Lu, Junhao Shi, Luca Jiang-Tao Yu, et al. Faster: Toward efficient autoregressive vision language action modeling via neural action tokenization. *arXiv preprint arXiv:2512.04952*, 2025.

- Run Luo, Xiaobo Xia, Lu Wang, Longze Chen, Renke Shan, Jing Luo, Min Yang, and Tat-Seng Chua. Next-omni: Towards any-to-any omnimodal foundation models with discrete flow matching. *arXiv preprint arXiv:2510.13721*, 2025.
- Corey Lynch and Pierre Sermanet. Language conditioned imitation learning over unstructured data. *arXiv preprint arXiv:2005.07648*, 2020.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters (RA-L)*, 7(3): 7327–7334, 2022.
- John Nguyen, Marton Havasi, Tariq Berrada, Luke Zettlemoyer, and Ricky TQ Chen. Oneflow: Concurrent mixed-modal and interleaved generation with edit flows. *arXiv preprint arXiv:2510.03506*, 2025.
- Octo Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, Tobias Kreiman, You Liang Tan, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- Moritz Reuss, Jyothish Pari, Pulkit Agrawal, and Rudolf Lioutikov. Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Neta Shaul, Itai Gat, Marton Havasi, Daniel Severo, Anuroop Sriram, Peter Holderrieth, Brian Karrer, Yaron Lipman, and Ricky TQ Chen. Flow matching with general discrete paths: A kinetic-optimal perspective. *arXiv preprint arXiv:2412.03487*, 2024.
- Wenxuan Song, Jiayi Chen, Pengxiang Ding, Yuxin Huang, Han Zhao, Donglin Wang, and Haoang Li. Ceed-vla: Consistency vision-language-action model with early-exit decoding. *arXiv preprint arXiv:2506.13725*, 2025a.
- Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Jun Ma, and Haoang Li. Accelerating vision-language-action model integrated with action chunking via parallel decoding. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025b.
- Wenxuan Song, Ziyang Zhou, Han Zhao, Jiayi Chen, Pengxiang Ding, Haodong Yan, Yuxin Huang, Feilong Tang, Donglin Wang, and Haoang Li. Reconvla: Reconstructive vision-language-action model as effective robot perceiver. *arXiv preprint arXiv:2508.10333*, 2025c.
- Wenxuan Song, Jiayi Chen, Xiaoquan Sun, Huashuo Lei, Yikai Qin, Wei Zhao, Pengxiang Ding, Han Zhao, Tongxin Wang, Pengxu Hou, Zhide Zhong, Haodong Yan, Donglin Wang, Jun Ma, and Haoang Li. Rethinking the practicality of vision-language-action model: A comprehensive benchmark and an improved baseline, 2026.
- Jin Wang, Yao Lai, Aoxue Li, Shifeng Zhang, Jiacheng Sun, Ning Kang, Chengyue Wu, Zhenguo Li, and Ping Luo. Fudoki: Discrete flow-based unified understanding and generation via kinetic-optimal velocities. *arXiv preprint arXiv:2505.20147*, 2025a.
- Xinlong Wang, Xiaosong Zhang, Zhengxiong Luo, Quan Sun, Yufeng Cui, Jinsheng Wang, Fan Zhang, Yueze Wang, Zhen Li, Qiyang Yu, et al. Emu3: Next-token prediction is all you need. *arXiv preprint arXiv:2409.18869*, 2024.
- Yating Wang, Haoyi Zhu, Mingyu Liu, Jiange Yang, Hao-Shu Fang, and Tong He. Vq-vla: Improving vision-language-action models via scaling vector-quantized action tokenizers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11089–11099, 2025b.

- Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, Siteng Huang, Yifan Tang, Wenhui Wang, Ru Zhang, Jianyi Liu, and Donglin Wang. Vla-adapter: An effective paradigm for tiny-scale vision-language-action model. *arXiv preprint arXiv:2509.09372*, 2025c.
- Yuqi Wang, Xinghang Li, Wenxuan Wang, Junbo Zhang, Yingyan Li, Yuntao Chen, Xinlong Wang, and Zhaoxiang Zhang. Unified vision-language-action model. *arXiv preprint arXiv:2506.19850*, 2025d.
- Junjie Wen, Minjie Zhu, Jiaming Liu, Zhiyuan Liu, Yicun Yang, Linfeng Zhang, Shanghang Zhang, Yichen Zhu, and Yi Xu. dvla: Diffusion vision-language-action model with multimodal chain-of-thought. *arXiv preprint arXiv:2509.25681*, 2025a.
- Yuqing Wen, Hebei Li, Kefan Gu, Yucheng Zhao, Tiancai Wang, and Xiaoyan Sun. Llada-vla: Vision language diffusion action models. *arXiv preprint arXiv:2509.06932*, 2025b.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.
- Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024.
- Haodong Yan, Zhide Zhong, Jiaguan Zhu, Junjie He, Weilin Yuan, Wenxuan Song, Xin Gong, Yingjie Cai, Guanyi Zhao, Xu Yan, Bingbing Liu, Ying-Cong Chen, and Haoang Li. S-vam: Shortcut video-action model by self-distilling geometric and semantic foresight, 2026. <https://arxiv.org/abs/2603.16195>.
- Jiacheng Ye, Shansan Gong, Jiahui Gao, Junming Fan, Shuang Wu, Wei Bi, Haoli Bai, Lifeng Shang, and Lingpeng Kong. Dream-vl & dream-vla: Open vision-language and vision-language-action models with diffusion language model backbone. *arXiv preprint arXiv:2512.22615*, 2025a.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025b.
- Runpeng Yu, Qi Li, and Xinchao Wang. Discrete diffusion in large language and multimodal models: A survey. *arXiv preprint arXiv:2506.13759*, 2025.
- Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37:56619–56643, 2024.
- Jianke Zhang, Yanjiang Guo, Yucheng Hu, Xiaoyu Chen, Xiang Zhu, and Jianyu Chen. Up-vla: A unified understanding and prediction model for embodied agent. *arXiv preprint arXiv:2501.18867*, 2025a.
- Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, et al. Dreamvla: a vision-language-action model dreamed with comprehensive world knowledge. *arXiv preprint arXiv:2507.04447*, 2025b.
- Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. pages 1702–1713, 2025.
- Chuanxia Zheng, Tung-Long Vuong, Jianfei Cai, and Dinh Phung. Movq: Modulating quantized vectors for high-fidelity image generation. *Advances in Neural Information Processing Systems*, 35:23412–23425, 2022.
- Zhide Zhong, Haodong Yan, Junfeng Li, Xiangchen Liu, Xin Gong, Wenxuan Song, Jiayi Chen, and Haoang Li. Flowvla: Thinking in motion with a visual chain of thought. *arXiv preprint arXiv:2508.18269*, 2025.
- Zhide Zhong, Junfeng Li, Junjie He, Haodong Yan, Xin Gong, Guanyi Zhao, Yingjie Cai, Jiantao Gao, Xu Yan, Bingbing Liu, Yingcong Chen, Liuqing Yang, and Haoang Li. Dualcot-vla: Visual-linguistic chain of thought via parallel reasoning for vision-language-action models, 2026. <https://arxiv.org/abs/2603.22280>.

Algorithm 2 Action-Modality DFM Training

Require: Predictor p_θ , learning rate η , optional weight μ

- 1: **repeat**
 - 2: Sample $(l, x_1) \sim p_{\text{data}}$
 - 3: Sample $t \sim \mathcal{U}(0, 1)$
 - 4: Sample $x_t \sim p_t(\cdot | x_1)$
 - 5: Choose $\mathcal{L}_{\text{train}} \in \{\mathcal{L}_{\text{ce}}, \mathcal{L}_{\text{head}}\}$
 - 6: Compute $\mathcal{L}_{\text{train}}$ from (x_t, l) and x_1
 - 7: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{train}}$
 - 8: **until** converged
 - 9: **return** trained predictor p_θ
-

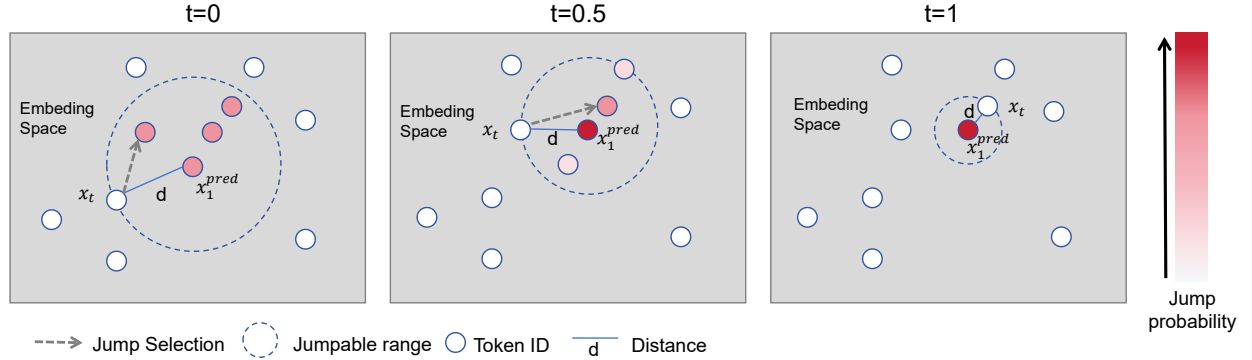


Figure S1 Embedding-space view of iterative refinement. From $t = 0$ to $t = 1$, the current token x_t progressively moves toward x_1^{pred} , while transition probabilities increasingly concentrate around the target.

Appendix

A Training Pipeline

Following the notation in the main manuscript, we denote the discretized instruction and observations as l , and apply noising and prediction only to the action modality. As shown in [Algorithm 2](#), let $x_1 = [x_1^1, \dots, x_1^D]$ be the clean action token sequence. At each training iteration, we sample $t \sim \mathcal{U}(0, 1)$, draw corrupted action tokens $x_t \sim p_t(\cdot | x_1)$ from the prescribed probability path, and feed (x_t, l) into the predictor to obtain per-position distributions $p_{1|t}^\theta(\cdot | x_t, l)$. The model refines all action tokens in parallel.

The training objective follows the same definitions in the main manuscript. Depending on the velocity modeling variant, we optimize either $\mathcal{L}_{\text{ce}}$ or $\mathcal{L}_{\text{head}}$.

B Visualization of Refinement Process

To provide intuition for the action-embedding-guided velocity modeling in the main manuscript, [Figure S1](#) visualizes a token refinement trajectory in the embedding space at three representative times ($t = 0$, $t = 0.5$, and $t = 1$). The hollow blue circles denote candidate token IDs, the red point denotes the predicted clean token x_1^{pred} , and x_t denotes the current token state.

At each step, the transition rate favors destinations that are closer to x_1^{pred} than the current token. As a result, tokens within the jumpable range can be selected, and candidates nearer to x_1^{pred} receive higher jump probability (deeper red). As refinement proceeds, x_t moves toward x_1^{pred} , the feasible jump set becomes progressively smaller, and updates become increasingly local. This behavior illustrates how DFM-VLA balances early exploration with late-stage stabilization during iterative action refinement.

Table S1 Implementation hyperparameters for training.

Parameter	Value
nproc_per_node	8
nnodes	1
learning_rate	8e-5
min_learning_rate	5e-6
lr_scheduler_type	cosine_with_min_lr
warmup_steps	50
weight_decay	0.1
max_grad_norm	5.0
adam_beta1 / adam_beta2	0.9 / 0.95
adam_epsilon	1e-6
bf16 / tf32	True / True
max_steps	16000
gradient_accumulation_steps	4
effective global batch size	8 * NGPUS * 4

C Implementation Hyperparameters

Table S1 summarizes the launch and training hyperparameters used in our implementation.